

Introduction to Query Optimization

LAB 1:

```
21
22 --lab 1
23 EXPLAIN (ANALYZE, BUFFERS)
24 SELECT c.first_name, c.last_name, r.rental_date
25 FROM customer c
26 JOIN rental r ON c.customer_id = r.customer_id
27 WHERE c.customer_id = 1;
```

Data Output Messages Notifications

QUERY PLAN

1	Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=0.943..4.449 rows=36.00 loops=1)
2	Buffers: shared hit=1 read=152 dirtied=1
3	-> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=0.472..0.476 rows=1.00 loops=1)
4	Index Cond: (customer_id = 1)
5	Index Searches: 1
6	Buffers: shared hit=1 read=2
7	-> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=0.467..3.957 rows=36.00 loops=1)
8	Filter: (customer_id = 1)
9	Rows Removed by Filter: 16011
10	Buffers: shared read=150 dirtied=1
11	Planning:
12	Buffers: shared hit=85 read=5
13	Planning Time: 2.447 ms
14	Execution Time: 4.489 ms

+ Bảng outer: customers

+ Bảng inner: rental

+ Hệ thống chọn Nested Loop vì: Tại dòng số 3, bảng customer sau khi lọc (customer_id = 1) trả về duy nhất 1 bản ghi (actual ... rows=1.00). Vì kích thước của bảng Outer quá nhỏ, thuật toán Nested Loop là tối ưu nhất. Nếu hệ thống chọn thuật toán Hash Join, nó sẽ phải tốn chi phí để cấp phát bộ nhớ RAM và xây dựng Bảng băm chỉ để lưu trữ đúng 1 dòng dữ liệu. Điều này là lãng phí và "quá đắt" so với việc sử dụng một vòng lặp đơn giản.

+ Hiệu quả: Nút quét bảng Inner ở dòng số 7, thông số ghi nhận là loops=1. Thuật toán Nested Loop quét bảng Inner (bảng rental) số lần bằng đúng với số dòng dữ liệu trả về từ bảng Outer. Do bảng customer chỉ trả về 1 dòng khách hàng, hệ thống chỉ cần thực hiện vòng lặp bên trong đúng 1 lần duy nhất để tìm kiếm các bản ghi thuê đĩa tương ứng.

LAB 2

```

28
29 --lab 2
30 EXPLAIN (ANALYZE, BUFFERS)
31 SELECT f.title, l.name as language
32 FROM film f
33 JOIN language l ON f.language_id = l.language_id;

```

Data Output Messages Notifications

QUERY PLAN

1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.605..4.054 rows=1000.00 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared hit=39 read=50
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.022..2.998 rows=1000.00 loops=1)
5	Buffers: shared hit=39 read=49
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.569..0.570 rows=6.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared read=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.546..0.548 rows=6.00 loops=1)
10	Buffers: shared read=1
11	Planning:
12	Buffers: shared hit=162 read=2
13	Planning Time: 3.159 ms
14	Execution Time: 4.220 ms

+ Xác định các giai đoạn (Identify Phases):

- Build (Bảng trong): Bảng language (do kích thước rất nhỏ, chỉ 6 dòng).
- Probe (Bảng ngoài): Bảng film (quét 1 lần để dò tìm với bảng băm).

+ Kiểm tra bộ nhớ (Memory Check): Nút Hash sử dụng 9kB (Batches: 1). Dữ liệu nằm hoàn toàn trên RAM, không bị ghi ra đĩa.

+ SET work_mem = '64kB';: Không có gì thay đổi, vì bộ nhớ cần thiết (9kB) vẫn nhỏ hơn giới hạn 64kB. Chưa đủ điều kiện để xảy ra hiện tượng tràn ổ đĩa (Spill to Disk).

```

30  --lab 2
31  SET work_mem = '64kB';
32  EXPLAIN (ANALYZE, BUFFERS)
33  SELECT f.title, l.name as language
34  FROM film f
35  JOIN language l ON f.language_id = l.language_id;
36

```

Data Output Messages Notifications

	QUERY PLAN
	text
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.200..1.116 rows=1000.00 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared hit=89
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.066..0.527 rows=1000.00 loo...
5	Buffers: shared hit=88
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.069..0.070 rows=6.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared hit=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.042..0.045 rows=6.00 loo...
10	Buffers: shared hit=1
11	Planning Time: 0.445 ms
12	Execution Time: 1.305 ms

LAB 3:

Data Output Messages Notifications

- Có xuất hiện nút Sort (Dòng 10) áp dụng cho bảng inventory. Hệ thống phải quét toàn bộ bảng (Seq Scan) và sắp xếp thủ công, thậm chí phải ghi ra ổ đĩa (Sort Method: external merge Disk: 80kB ở dòng 12).
- Tuy nhiên, bảng film lại sử dụng Index Scan (Dòng 4) nên dữ liệu đã được sắp xếp sẵn mà không cần nút Sort (được "miễn phí" chi phí sắp xếp).
- Nếu gỡ ORDER BY và RESET ALL?
 - o Hệ thống sẽ chuyển lại thành thuật toán Hash Join.
 - o *Lý do:* Vì bảng inventory không có sẵn Index phù hợp để sắp xếp, việc ép dùng Merge Join khiến hệ thống bị tốn chi phí (cost) cho quá trình Sort thủ công. Khi không bắt buộc phải trả kết quả theo thứ tự (ORDER BY), Hash Join sẽ là lựa chọn rẻ và nhanh hơn.

HANDS-ON-LAB

Task 1:

```

47  -- task 1:
48  SET enable_indexscan = off;
49  EXPLAIN ANALYZE SELECT * FROM rental WHERE rental_date > '2005-05-24';

```

Data Output Messages Notifications

QUERY PLAN text

1	Seq Scan on rental (cost=0.00..350.59 rows=16047 width=36) (actual time=0.022..3.558 rows=16047.00 loop...
2	Filter: (rental_date > '2005-05-24 00:00:00'::timestamp without time zone)
3	Buffers: shared hit=150
4	Planning:
5	Buffers: shared hit=23 read=3 dirtied=1
6	Planning Time: 1.613 ms
7	Execution Time: 4.923 ms

```

50
51  SET enable_indexscan = on;
52  EXPLAIN ANALYZE SELECT * FROM rental WHERE rental_date > '2005-05-24';

```

Data Output Messages Notifications

QUERY PLAN text

1	Seq Scan on rental (cost=0.00..350.59 rows=16047 width=36) (actual time=0.019..3.185 rows=16047.00 loop...
2	Filter: (rental_date > '2005-05-24 00:00:00'::timestamp without time zone)
3	Buffers: shared hit=150
4	Planning:
5	Buffers: shared hit=4
6	Planning Time: 0.209 ms
7	Execution Time: 4.538 ms

- Khi Tắt (OFF): Hệ thống dùng Seq Scan, Cost = 350.59, Thời gian thực thi = 4.923 ms.
- Khi Bật (ON): Hệ thống vẫn dùng Seq Scan, Cost = 350.59, Thời gian thực thi = 4.538 ms.
- Dù bật (ON) hay tắt (OFF) tham số enable_indexscan, Optimizer vẫn quyết định sử dụng Seq Scan (quét toàn bộ bảng).

Task 2:

```

53
54 --task 2:
55 EXPLAIN (ANALYZE, BUFFERS)
56 SELECT c.first_name, c.last_name, r.rental_date, i.inventory_id
57 FROM customer c
58 JOIN rental r ON c.customer_id = r.customer_id
59 JOIN inventory i ON r.inventory_id = i.inventory_id;

```

Data Output
Messages
Notifications

	QUERY PLAN
1	Merge Join (cost=3018.53..3262.23 rows=16047 width=25) (actual time=29.839..33.285 rows=16047.00 loops=1)
2	Merge Cond: (r.customer_id = c.customer_id)
3	Buffers: shared hit=16078 read=37 dirtied=2, temp read=108 written=121
4	-> Sort (cost=2975.91..3016.03 rows=16047 width=14) (actual time=29.626..30.865 rows=16047.00 loops=1)
5	Sort Key: r.customer_id
6	Sort Method: external merge Disk: 448kB
7	Buffers: shared hit=16069 read=37 dirtied=2, temp read=108 written=121
8	-> Merge Join (cost=0.57..1302.03 rows=16047 width=14) (actual time=0.049..14.863 rows=16047.00 loops=1)
9	Merge Cond: (r.inventory_id = i.inventory_id)
10	Buffers: shared hit=16069 read=37 dirtied=2
11	-> Index Scan using idx_fk_inventory_id on rental r (cost=0.29..960.99 rows=16047 width=14) (actual time=0.020..8.911 rows=16047.00 loops=1)
12	Index Searches: 1
13	Buffers: shared hit=16056 read=26 dirtied=1
14	-> Index Only Scan using inventory_pkey on inventory i (cost=0.28..129.00 rows=4581 width=4) (actual time=0.025..1.443 rows=4581.00 loops=1)
15	Heap Fetches: 329
16	Index Searches: 1
17	Buffers: shared hit=13 read=11 dirtied=1
18	-> Sort (cost=42.62..44.12 rows=599 width=17) (actual time=0.210..0.234 rows=599.00 loops=1)
19	Sort Key: c.customer_id
20	Sort Method: quicksort Memory: 47kB
21	Buffers: shared hit=9
22	-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=17) (actual time=0.013..0.088 rows=599.00 loops=1)
23	Buffers: shared hit=9
24	Planning:
25	Buffers: shared hit=28 read=9 dirtied=2
26	Planning Time: 4.133 ms
27	Execution Time: 34.266 ms

- Hệ thống sử dụng Merge Join cho toàn bộ quá trình kết hợp 3 bảng (rental, inventory và customer).
- Quá trình join rental và inventory đạt tối ưu nhờ Index đã sắp xếp sẵn. Tuy nhiên, khi join kết quả với bảng customer, hệ thống phải thực hiện lệnh Sort thủ công và bị ghi tràn bộ nhớ ra ổ đĩa (Disk: 448kB), tạo ra nút thắt cổ chai nhỏ.

```

53
54 --task 2:
55 ANALYZE customer, rental, inventory;
56
57 EXPLAIN (ANALYZE, BUFFERS)
58 SELECT c.first_name, c.last_name, r.rental_date, i.inventory_id
59 FROM customer c
60 JOIN rental r ON c.customer_id = r.customer_id
61 JOIN inventory i ON r.inventory_id = i.inventory_id;

```

Data Output Messages Notifications

SQL

QUERY PLAN	
text	
1	Merge Join (cost=3023.54..3267.24 rows=16047 width=25) (actual time=29.884..31.404 rows=16047.00 loops=1)
2	Merge Cond: (r.customer_id = c.customer_id)
3	Buffers: shared hit=16099, temp read=108 written=121
4	-> Sort (cost=2980.92..3021.04 rows=16047 width=14) (actual time=29.701..30.266 rows=16047.00 loops=1)
5	Sort Key: r.customer_id
6	Sort Method: external merge Disk: 448kB
7	Buffers: shared hit=16090, temp read=108 written=121
8	-> Merge Join (cost=0.57..1307.04 rows=16047 width=14) (actual time=0.047..14.787 rows=16047.00 loops=1)
9	Merge Cond: (r.inventory_id = i.inventory_id)
10	Buffers: shared hit=16090
11	-> Index Scan using idx_fk_inventory_id on rental r (cost=0.29..960.99 rows=16047 width=14) (actual time=0.016..8.510 rows=16047.00 loops=1)
12	Index Searches: 1
13	Buffers: shared hit=16072
14	-> Index Only Scan using inventory_pkey on inventory i (cost=0.28..134.01 rows=4581 width=4) (actual time=0.026..0.776 rows=4581.00 loops=1)
15	Heap Fetches: 326
16	Index Searches: 1
17	Buffers: shared hit=18
18	-> Sort (cost=42.62..44.12 rows=599 width=17) (actual time=0.180..0.192 rows=599.00 loops=1)
19	Sort Key: c.customer_id
20	Sort Method: quicksort Memory: 47kB
21	Buffers: shared hit=9
22	-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=17) (actual time=0.010..0.043 rows=599.00 loops=1)
23	Buffers: shared hit=9
24	Planning:
25	Buffers: shared hit=94
26	Planning Time: 2.862 ms
27	Execution Time: 32.022 ms

- Sau khi chạy ANALYZE cho 3 bảng, cấu trúc Query Plan không thay đổi (vẫn dùng Merge Join), chỉ có chi phí (Cost) thay đổi rất nhẹ (từ 3262.23 lên 3267.24). Lý do là số liệu thống kê trong pg_stats đã được làm mới, nhưng cấu trúc dữ liệu thực tế không có biến động lớn nên Optimizer vẫn giữ nguyên kế hoạch cũ.